



BLUE BITS

البرمجة التفرعية

نظري (7)

ملف داعم

2026/2025

السنة الرابعة



مكتبة شهاب
للشعاع النيرة - يوق يلقى على القاصي

[f](#) [@](#) [v](#) [t](#) /BlueBitsTeam

www.bluebitsteam.com

تقنية فرق تسد *Divide – and – conquer*

- تتميز هذه التقنية بتقسيم مشكلة ضخمة إلى مشاكل فرعية أصغر من نفس شكل المشكلة الأكبر
 - عادة ما يتم إجراء العديد من التقسيمات إلى مشكلات فرعية أصغر عن طريق التعاودية
 - سوف تقوم الطريقة العودية بتقسيم المشكلة باستمرار حتى لا تتمكن من تقسيم المهام إلى أجزاء أصغر.
 - يتم تنفيذ المهام البسيطة جداً ودمج النتائج مع استمرار في الجمع مع مهام أكبر وأكبر
 - أي أن الجزء الأول من فرق تسد تقسيم العمل على عدة *Processes* مختلفة والجزء الثاني إعادة تجميع البيانات المرسلة على ال *Processes*
 - نقوم بتقسيم العمل عندما:
 1. تكون حجم البيانات كبيرة
 2. عندما نريد تحديد من سينجز العمل (عدد المعالجات التي ستقوم بإنجاز العمل التفرعي).
 - عدد ال *Processes* الذي يجب اعتماده من أجل التقسيم يكون حسب عدد العناصر ويجب أن تكون ضخمة (تقتضي التقسيم)
- مثال:** التعريف العودي المتسلسل لإضافة قائمة من الأرقام.

مثال:

التعريف التكراري المتسلسل لإضافة قائمة من الأرقام هو:

```

1  int add(int * s) /* add list of numbers, s */
2  {
3  if (number(s) <= 2) return (n1 + n2);/* see explanation */
4  else {
5  Divide (s, s1, s2);/* divide s into two parts, s1 and s2 */
6  part_sum1 = add(s1);/* recursive calls to add sub lists */
7  part_sum2 = add(s2);
8  return (part_sum1 + part_sum2);
9  }
10 }
```

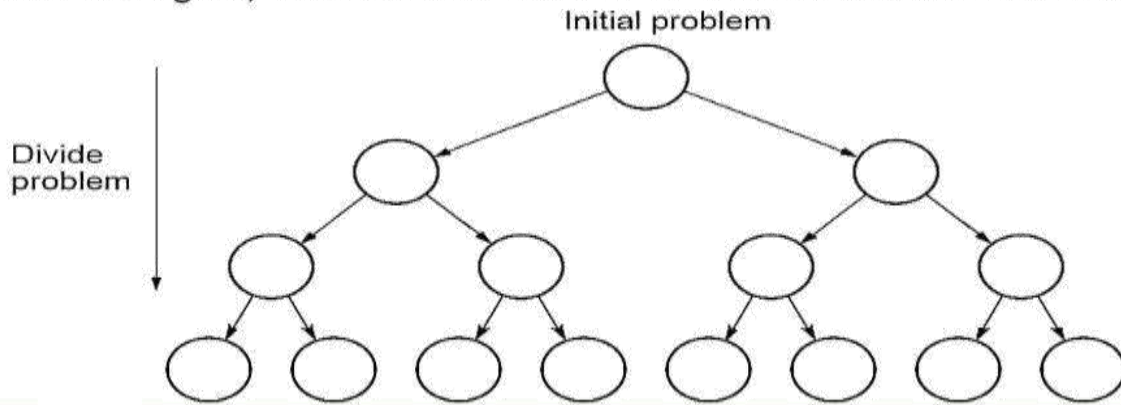
- في جميع التعاريف العودية يجب أن يكون هناك طريقة لإنهاء العودية عندما لا نستطيع التقسيم لأجزاء أصغر.
 - في البرنامج السابق، التابع *number(s)* يرجع عدد الأرقام الموجود في القائمة *number(S)* المؤشر عليها *S*
 - إذا كان هناك رقمان في القائمة فإنهما يسميان ب *n1, n2*
 - إذا كان هناك رقم واحد في القائمة يسمى *n1* و *n2* يكون صفراً
 - إذا لم يكن هناك أرقام فإن كلا *n1, n2* يساوي صفراً
- يمكن استخدام هذه الطريقة لعمليات أخرى في القائمة مثل:
1. إيجاد أكبر عدد *find the maximum number*
 2. فرز القائمة من خلال تقسيمها إلى قوائم أصغر فأصغر للفرز *Sorting a list*



- في كل مرحلة تقسيم، عندما ينشئ كل قسم جزأين، طريقة فرق تسد العودية تشكّل شجرة ثنائية
- يتم اجتياز الشجرة نحو الأسفل عند إجراء الاستدعاءات، ونحو الأعلى عند استعادة الاستدعاءات
- عندما يكون لدينا عمل ضخم نقسمه إلى عدد من *Processes* أصغر وبنفس الوقت نستطيع اعتماد أسلوب الشجرة كحل للمشكلة.

Divide and Conquer: Example (cont.)

A binary tree construction showing the "divide" part of divide and conquer is shown in this figure, with the final tasks at the bottom and the root at the top.



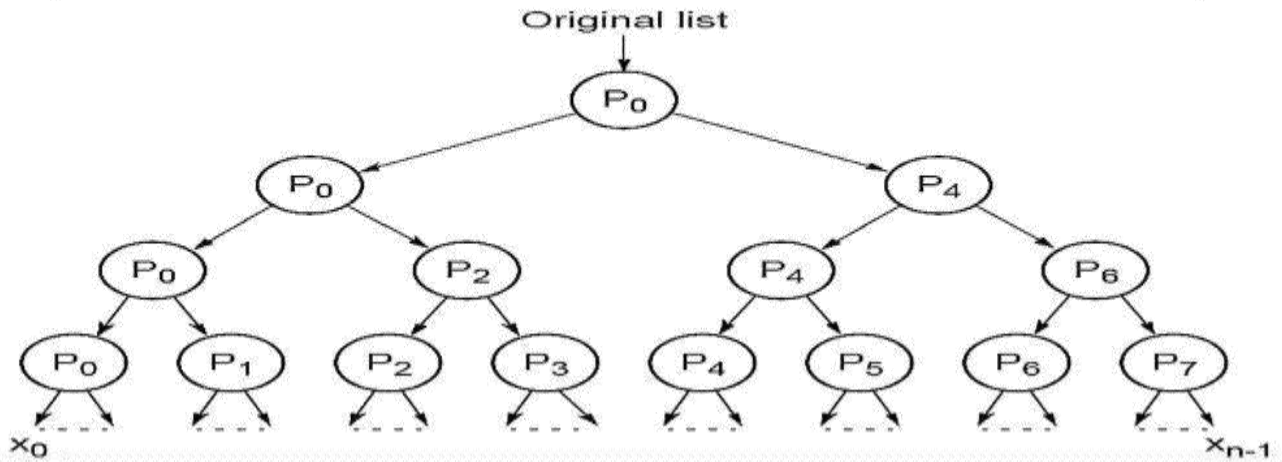
- عملية الجذر تقسم المشكلة إلى قسمين
- وينقسم كل من هذين الجزئين إلى قسمين وهكذا حتى نصل إلى الأوراق
- يمكن استخدام هذا الأسلوب في بناء الشجرة في المشكلة السابقة:
- 1. لتقسيم قائمة الأرقام أولاً إلى قسمين ثم إلى أربعة أقسام وهكذا حتى يكون لكل عملية جزء متساوٍ من الكل
- 2. بعد إضافة الأزواج في أسفل الشجرة يحدث التراكم في بناء شجرة عكسية
- يوضح الشكل السابق شجرة ثنائية كاملة متوازنة مع جميع العقد السفلية على نفس المستوى
- يحدث هذا إذا كان بالإمكان تقسيم المهمة إلى عدد من الأجزاء قابل للرفع إلى القوة 2
- إذا لم تكن قابلة للرفع إلى القوة 2 فيكون هناك عقدة سفلية واحدة أو أكثر في مستوى واحد أعلى من العقد الأخرى (الأوراق)
- سنفترض دائماً أنّ المهمة يمكن تقسيمها إلى عدد من الأجزاء بالقوة 2 ، إلّا إذا تم ذكر غير ذلك

في التنفيذ التسلسلي، لا يمكننا سوى زيارة عقدة واحدة من الشجرة في كل مرة.

يوفر الحل المتوازي / التفرعي إمكانية اجتياز عدة أجزاء من الشجرة في وقت واحد. فبمجرد تقسيم الشجرة إلى جزأين، يمكن معالجة كلا الجزأين في وقت واحد.

يكمّن الموضوع في إدراك أن البنية عبارة عن شجرة

Parallel Implementation (cont.)

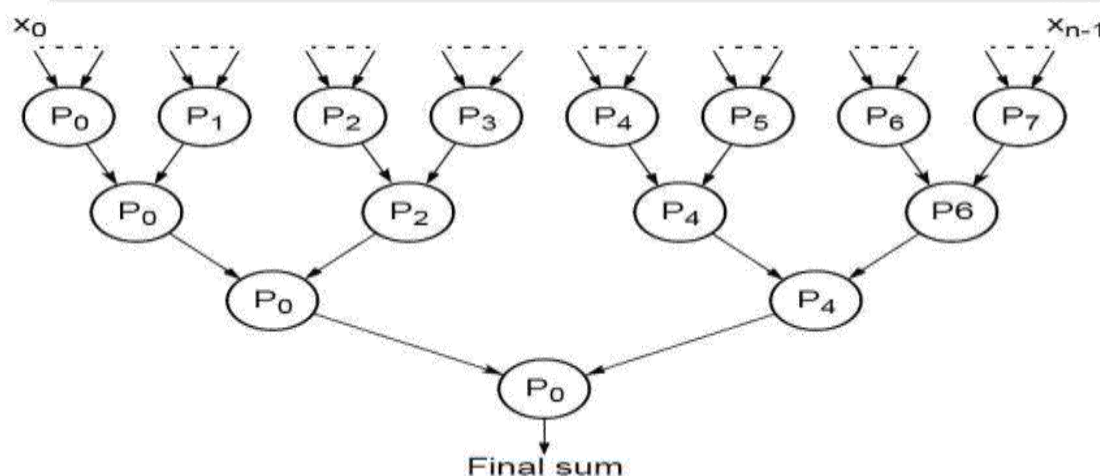


- يمكن تعيين معالج واحد لكل عقدة في الشجرة وسيطلب ذلك في النهاية معالجات عددها $2^{(m+1)}$ معالجة وذلك لتقسيم المهام إلى 2^m جزء
- سيكون كل معالج نشطاً فقط على مستوى واحد في الشجرة ويؤدي ذلك إلى حل غير فعال للغاية
- الحل الأكثر كفاءة هو إعادة استخدام المعالجات في كل مستوى من مستويات الشجرة والموضح في الشكل القادم
- الحل في الشكل السابق يستخدم ثمانية معالجات
- يتوقف التقسيم عند اكتمال عدد المعالجات
- في كل مرحلة يحتفظ كل معالج بنصف القائمة ويمرر النصف الآخر
 - أولاً يتواصل P_0 مع P_4 ويمرر نصف القائمة إلى P_4
 - ثم يقوم P_0 و P_4 بتمرير نصف من القائمة التي يحملانها إلى P_2 و P_6 على التوالي
 - أخيراً يمرر P_0 و P_2 و P_4 و P_6 نصف من القائمة التي يحملونها إلى P_1 و P_3 و P_5 و P_7 على التوالي
 - كل قائمة في المرحلة النهائية ستحتوي على $n/8$ أرقام أو n/p في الحالة العامة من عدد p من المعالجات
- توجد مستويات عددها $\log p$ في الشجرة
- في كل شجرة نريد:
 - عدد المستويات $m = \log p$
 - عدد الأوراق = عدد المعالجات $= 2^m$
 - عدد العقد $= 2^{(m+1)} - 1$

مثال:

- $\log_2 8 = \log_2 2^3 = 3 \log_2 2 = 3$ عدد المستويات
- $8^1 = 2^3$ عدد الأوراق = عدد المعالجات
- $15 = 2^4 - 1$ عدد العقد وكل عقدة داخلية تأخذ اسم ابنها اليساري

Parallel Implementation (cont.)



- يمكن القيام بعملية الدمج لجمع المجاميع الجزئية كما هو موضح في الشكل السابق
- لمجرد تكوين المجاميع الجزئية كل منها يحمل رقم فردي يقوم المعالج بتمرير مجموعة جزئية إلى الرقم الزوجي المجاور للمعالج

أي أن P_1 يمرر مجموعة إلى P_0 و P_3 إلى P_2 و P_5 إلى P_4 وهكذا.

- تقوم المعالجات ذات الأرقام الزوجية بعد ذلك بإضافة المجموع الجزئي إلى مجموعها الجزئي الخاص بها وتمرير النتيجة تصاعدياً (إلى الأمام) كما هو مبين
- يستمر ذلك حتى تحصل P_0 على النتيجة النهائية.

مثال:

(نظام تمرير الرسائل)

- في نظام تمرير الرسائل، يجب تمرير الأرقام إلى المعالجات بشكل فردي.
- في نظام الذاكرة المشتركة، يمكن لكل معالج الوصول إلى الأرقام التي يريدها من الذاكرة المشتركة، ومن هذا المنطلق، سيكون نظام الذاكرة المشتركة أكثر ملاءمة لهذه المشكلة والمشاكل المماثلة

Parallel Implementation (cont.)

The parallel code for process P_0 might take the form:

```

Process  $P_0$ 

divide(s1, s1, s2);          /* division phase */
send(s2,  $P_1$ );              /* divide s1 into two, s1 and s2 */
divide(s1, s1, s2);          /* send one part to another process */
send(s2,  $P_2$ );
divide(s1, s1, s2);
send(s2,  $P_1$ );
part_sum = add(s1);          /* combining phase */
recv(&part_sum1,  $P_1$ );
part_sum = part_sum + part_sum1;
recv(&part_sum1,  $P_2$ );
part_sum = part_sum + part_sum1;
recv(&part_sum1,  $P_4$ );
part_sum = part_sum + part_sum1;

```

شرح:

(مرحلة التقسيم): في العملية P_0 (الجزر) تقوم بتقسيم المهمة إلى قسمين عن طريق تابع *divide* حيث تحتفظ بالقسم الأول وترسل القسم الثاني إلى العملية P_4 ، ثم تتابع بتقسيم قسمها الأول إلى قسمين وإرسال القسم الثاني إلى العملية P_1 . تستمر العملية P_0 في تقسيم قسمها حتى لا تتمكن من تقسيمه أبداً.

(مرحلة التجميع): نعلم أنّ النتيجة النهائية سيتم تخزينها في الجزر، لذلك نعرّف متحول نضع فيه مجموع الجزء الخاص بالعملية P_0 ، ثم سوف تستقبل عن طريق تابع *recv* المجموع الجزئي الخاص بالعملية P_1 وتقوم بإضافته إلى مجموع قسمها، ثم تستقبل المجموع الجزئي الخاص بالعملية P_2 ثم تقوم بإضافته مع المجموع الجزئي السابق وهكذا حتى تحصل على النتيجة النهائية

Parallel Implementation (cont.)

The parallel code for process P_4 might take the form:

```

Process  $P_4$ 

recv(s1,  $P_0$ );              /* division phase */
divide(s1, s1, s2);
send(s2,  $P_6$ );
divide(s1, s1, s2);
send(s2,  $P_5$ );
part_sum = add(s1);          /* combining phase */
recv(&part_sum1,  $P_5$ );
part_sum = part_sum + part_sum1;
recv(&part_sum1,  $P_6$ );
part_sum = part_sum + part_sum1;
send(&part_sum,  $P_0$ );

```

تمر العملية P_4 بنفس مراحل العملية P_0 لكن عندما تنتهي من تجميع قسمها تقوم بإرساله إلى العملية P_0 ، كون الناتج الأخير يكون عندها في البرنامج السابق، يتم تقسيم المهام بين العمليّات باستخدام *send* و *receive*

ملاحظة مهمة:

المثال السابق هام جداً ويُعد من القضايا الصعبة ويأتي بالامتحان بالامتحان تفترض لنا اسم التوابع وبارامترات الدخل والقيمة المعادة من بارامترات الدخل

التحليل : Analysis

- سنفترض أن n هو في القوة 2.
- وقت إعداد الاتصال: *t startup* "غير مضمن فيما يلي للتبسيط"
- تتكون مرحلة التقسيم *dividing phase* أساساً من التواصل فقط إذا افترضنا أن تقسيم القائمة إلى جزأين يتطلب الحد الأدنى من الحسابات.
- تتطلب مرحلة الدمج *Combining phase* الحساب والتواصل لإضافة المجاميع الجزئية *partial sum* المستلمة وتمرير النتيجة.

التواصل : Communication

يوجد عدد لوغاريتمي من الخطوات في مرحلة التقسيم وهو $\log p$ من الخطوات مع عمليات عددها p

- يتم تحديد وقت الاتصال *Communication setup time* لهذه المرحلة بواسطة:
- $$t_{comm1} = \frac{n}{2} t_{data} + \frac{n}{4} t_{data} + \frac{n}{8} t_{data} + \dots + \frac{n}{p} t_{data} = \frac{n(p-1)}{p} t_{data}$$
- حيث t_{data} وقت الإرسال لكلمة واحدة فقط من البيانات *one data word*
 - يُعد الوقت t_{comm1} أفضل قليلاً من *simple broadcast*

تذكرة:

مجموع حدود متتالية هندسية: $S = \frac{a_0(1-q^n)}{1-q}$
حيث: a_0 الحد الأول، q أساس المتتالية.

$$\begin{aligned}
t_{comm1} &= \frac{n}{2} t_{data} + \frac{n}{4} t_{data} + \frac{n}{8} t_{data} + \dots + \frac{n}{p} t_{data} \\
&= n t_{data} \left(\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots + \frac{1}{p} \right) \\
&= n t_{data} \left(\frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \dots + \frac{1}{p} \right) \\
&= n t_{data} \left(\frac{1}{2} \left(\frac{1 - \frac{1}{p}}{1 - \frac{1}{2}} \right) \right) \rightarrow \text{يمثل الحد } q^n \\
&= n t_{data} \left(\frac{1}{2} \left(\frac{1 - \frac{1}{p}}{\frac{1}{2}} \right) \right) \\
&= n t_{data} \left(1 - \frac{1}{p} \right) \\
&= n t_{data} \left(\frac{p-1}{p} \right) \\
&= \frac{n(p-1)}{p} t_{data}
\end{aligned}$$

تُعد مرحلة الدمج *combining* مشابهة لمرحلة التواصل *communication*، باستثناء أنه يتم إرسال عنصر بيانات واحد فقط في كل رسالة

- (المجموع الجزئي):

$$t_{comm2} = (\log p) t_{data}$$

- إجمالي وقت الاتصال:

$$t_{comm} = t_{comm1} + t_{comm2} = \frac{n(p-1)}{p} t_{data} + (\log p) t_{data}$$

أو التعقيد الزمني $O(n)$ لعدد محدد من المعالجات

الحساب *Computation*:

الحساب: في نهاية مرحلة التقسيم، يتم جمع الأعداد n/p معًا، ثم تحدث إضافة واحدة في كل مرحلة خلال مرحلة الدمج.

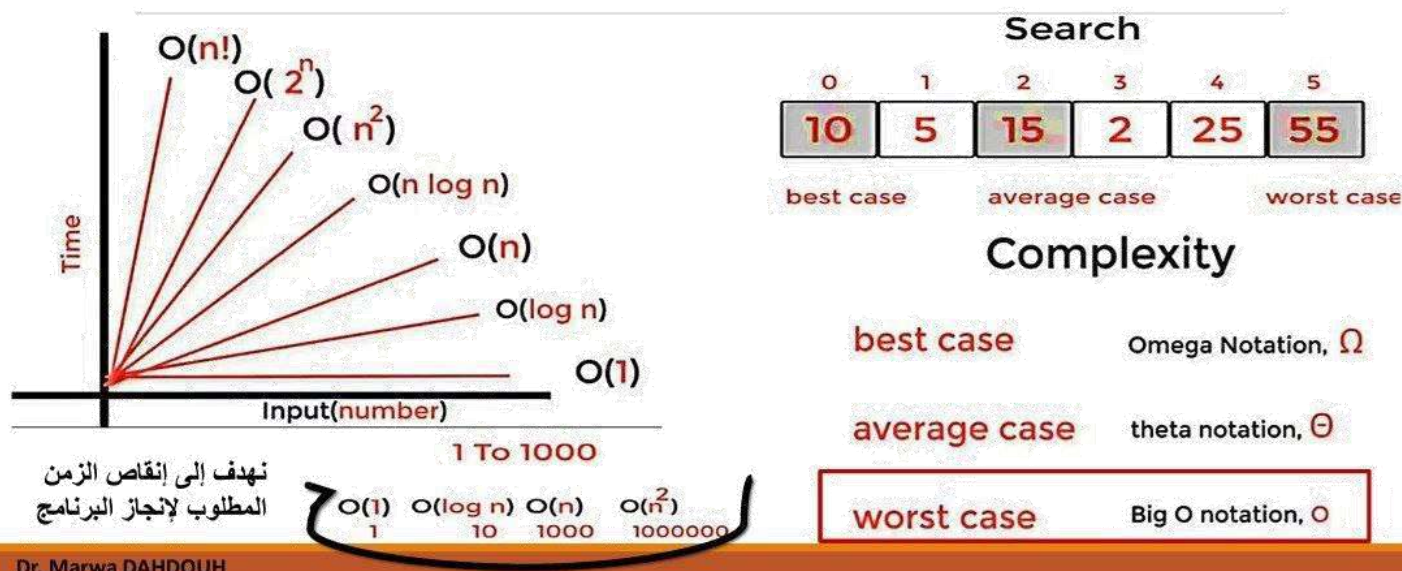
$$\text{leading to : } t_{comp} = (n/p) - 1 + \log p$$

أو مرة أخرى التعقيد الزمني $O(n)$ لعدد ثابت من المعالجات من أجل n عدد كبير و P متغير نحصل على $O(\frac{n}{p})$ وقت التنفيذ الإجمالي: يصبح إجمالي وقت التنفيذ التفرعي:

$$t_p = \left(\frac{n(p-1)}{p} + \log p \right) t_{data} + \frac{n}{p} + \log p$$



Complexity



Complexity

درجة تعقيد
العمليات البسيطة

$*, /, \%, ^, +, -$

$++, --$

$+=, -=, /=, *=$

if, else, ifelse

constant time

درجة تعقيد الحلقات
التكرارية

Complexity

```
sum;
for(i = 1; i <= n; i++) // n
    sum = sum + i; // constant time 1
```

Time Complexity : $1 + n$

برمجياً يمكن الاستغناء عن الحلقة التكرارية بعلاقة تُعطي
النتيجة مباشرة مما يؤثر على درجة التعقيد

$$n * (n + 1) / 2$$

$$5 * (5 + 1) / 2 = 15 \quad O(1)$$

$$1 + 2 + 3 + 4 + 5 = 15$$



درجة تعقيد حلقة وحيدة	درجة تعقيد حلقتين متداخلتين	درجة تعقيد ثلاث حلقات
<pre> 0 Int i; 1 i = 0; /* 1 */ 2 for(i=0; i<n; i++){ 3 /* n */ 4 print i;} </pre> $1 + n = O(n)$	<pre> 1 int i; 2 i=0; /* 1 */ 3 for(i=0; i<n; i++){ /* n 4 */ 5 print i; 6 for(j=0; j<n; j++) 7 /* n */ 8 for(int k=0; k<n; k++) 9 /* n */ Print j + k; </pre> $1 + n + (n * n) = O(n^2)$	<pre> 1 int i; 2 i=0; /* 1 */ 3 for(i=0; i<n; i++){ /* n 4 */ 5 print i; 6 for(j=0; j<n; j++) 7 /* n */ 8 for(int k=0; k<n; k++) 9 /* n */ 10 for(int l=1; l<n; l++) /* n */ </pre> $1 + (n * n * n) = O(n^3)$

درجة التعقيد عند خطوة زيادة محددة بعملية جداء أو قسمة تكون من مرتبة لوغاريتمية، يُعتبر مقدار الزيادة هو أساس للحد اللوغاريتم لذلك في المثال اللوغاريتم بأساس (2) بينما يصبح الأساس (3) إذا كان مقدار الزيادة ثلاثة

<pre> 6 int i; 7 i=1; /* 1 */ 8 for(i ; i<n ; i=i*2) 9 /* log n */ 10 print i; </pre> $1 + \log n = O(\log_2 n)$	<pre> 9 int i,j; 1 i = j = 0; /* 1 */ 2 for(i; i<n; i++){ /* n 3 */ 4 for(j; j<n; j = j/3) 5 /* n */ Print i+j; </pre> $1 + n * \log n = O(n \log_3 n)$	<pre> 5 for(int i=0; i<n; i++) 6 /* n */ 7 for(int j=0; j<n; j++) 8 for(int k=0; k<n; k=k*2) 9 print i+j+k; </pre> $n * n * \log n = n^2 \log n = O(n^2 * \log n)$
---	--	--

<pre> 7 for(int i=n/2; i<n; i++) 8 /* n/2 */ 9 for(int k=0; k<n; k=k*2) 10 /* log n */ 11 for(int j=0; j<n; j=2*j) 12 /* log n */ 13 print i+k+j; </pre> $\frac{n}{2} * \log n * \log n = n * \log n * \log n = n * \log^2 n = O(n * \log_2 n^2)$	<pre> 1 while(n>2)/* O(logn) 2 */ 3 . 4 . 5 . 6 n = n/2 </pre>	Loop: n Loop: n Loop: n Loop: n $O(n^4)$
---	---	--

*Complexity of Recursive*

درجة التعقيد <i>fibon</i>	درجة تعقيد مجموع أعداد	درجة تعقيد تابع تعاودي	درجة تعقيد تابع ما
2 int fib(int n) /* 4 */ 3 if(n<2) 4 return n; 5 return fib(n-1) + fib(n-2)	100 50 25 12.5 6.25 3.125 1.5625	7 fun(double n){ 8 if(n<1) 9 return; 0 print n; 1 fun(n/2);} $O(\log_2 n)$	4 for(int i=0; i<n; i++) 5 /* n */ 6 fun();

2026/2025

السنة الرابعة



BLUE BITS



أعضاء الفريق



الفريق التقني



الفريق العلمي

إبراهيم هبرة	عائشة العُمر العلّوش	إيناس بيلوني	ظلال بابا
إسراء حاج حمدي	عمر قظان	تسنيم طالب	عائشة قلّه
بتول لبّان	فاطمة الحسين	حلا إبراهيم	عائشة مُصطفى
بشرى كوكش	قاسم يحيى	دلال تسقيه	قمر كمد
بنيامين حكمان	كريست ياغجيان	ديانا صبّاغ	محمد يمان جلال
تسنيم حوكان	محمد منير	زهير حّقامي	محمود غزال
تسنيم نور الدّين	مديحة جّمّال	شكرية سُلطان	وفاء حّقامي
جميل هلال	ميرا فضلي	شهد كرّوم	يوسف نيّال
جودي جلال	نجم الدّين درويش		
خديجة بيازيد	هبة الله كتلو		
رشا معراوي	وداد قلعيّة		
سدرة مبيض	ياسر صبّاغ		

هل لديك أي ملاحظة؟ لا تتردّد في مراسلتنا.

